

AI Principles and Practice · COMP-0484

Supervised Learning: Regression and Classifica- tion

Week 2 · Reference Booklet

A plain-English guide: theory, mathematics, runnable Python, and outputs

Lecturer: Jonathan

How to use this booklet

Welcome back. This booklet is the long, friendly companion to the Week 2 lecture. The slides move fast; this document slows everything down and explains each idea as if we were sitting together at a whiteboard. Last week we learned to understand and prepare data. This week we use it to make predictions, and, just as importantly, we learn how to tell an honest result from a flattering one.

Every topic is built the same way, so you always know what to expect:

1. **A plain explanation** of what the idea is and why anyone should care about it.
2. **The mathematics**, written out as a formula, immediately followed by a sentence that reads the formula aloud in words.
3. **A short Python program** you can actually run, with a note on what each part does.
4. **The output**, shown in a dark box, with a short note on how to read it.
5. **A picture**, whenever a picture explains more than a paragraph could.

Everything here is reproducible. A single script, `make_assets.py`, recreates every number and every figure you see, using a fixed “random seed” so the results match this booklet exactly. The dataset, and the leakage-safe `prep` pipeline the code relies on, are the ones you built in Week 1; we simply hang two prediction jobs off them.

Setting up the tools (once)

You need Python and the same handful of libraries as last week. Nothing exotic is added: the gradient-boosting model we use later, `HistGradientBoosting`, already ships inside `scikit-learn`.

terminal

python

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pandas scikit-learn scipy matplotlib
python make_assets.py          # recreates every figure and printed output
```

What the libraries are for

NumPy does fast maths on arrays of numbers. **pandas** holds our table of data. **scikit-learn** is the machine-learning toolkit that supplies every model, metric, and the pipeline. **SciPy** adds statistics. **Matplotlib** draws the charts.

Contents

How to use this booklet	2
Setting up the tools (once)	2
1 From data to predictions	4
1.1 What supervised learning is	4
1.2 Train, validation and test	4
1.3 Regression or classification	5
1.4 The running example, now with two targets	5
2 Regression	6
2.1 Linear regression	6
2.2 Polynomial features and capacity	6
2.3 Regularisation: ridge and lasso	7
2.4 Fitting and comparing in code	7
2.5 Diagnosing a fit: residuals and R^2	8
3 Classification	10
3.1 Logistic regression	10
3.2 k-nearest neighbours	10
3.3 Decision trees	11
3.4 Ensembles: bagging and boosting	11
3.5 Support vector machines	11
3.6 Fitting and comparing in code	11
4 Loss, and the bias-variance trade-off	13
4.1 Loss functions	13
4.2 Under- and overfitting	13
4.3 The bias-variance decomposition	13
4.4 Learning curves	14
5 Evaluation done properly	15
5.1 Cross-validation	15
5.2 Regression metrics	15
5.3 The accuracy trap	16
5.4 The confusion matrix	16
5.5 Precision, recall and F1	17
5.6 ROC and AUC	17
5.7 Precision-recall for rare positives	18
5.8 Choosing the threshold	18
6 Framing real problems	20
6.1 The question sets the target, the target sets the metric	20
6.2 A tour of applications	20
6.3 Cost-sensitive thinking: two opposite problems	20
6.4 From a prediction to a decision	20
6.5 A framing checklist	20
A Formula cheat-sheet	22
B Running everything yourself	23
C Glossary	24

1 From data to predictions

Last week ended with a clean, documented dataset and a pipeline that prepares it without cheating. That is the launch pad. Supervised learning is the step where we stop describing the data and start using it to answer a question about a case we have not seen yet. Before any algorithm, it is worth being precise about what that means, because most mistakes are made here, not in the modelling.

1.1 What supervised learning is

In supervised learning we have, for each past case, both the inputs and the answer. We use those pairs to learn a rule that turns inputs into an answer, so that when a new case arrives with only its inputs, we can predict its answer.

$$\hat{y} = f(x; \theta)$$

the features x go in, a prediction $\hat{y}$ comes out; learning means choosing the parameters θ so the two agree on data we have not yet seen

Read this as: a model is a function f that takes the things we know about a case, written x , and produces a prediction, written $\hat{y}$ (said “y-hat”). The knobs inside that function are the parameters θ (“theta”), and training is nothing more than turning those knobs until the predictions line up with the known answers.

In plain terms

Think of it like a student revising from past exam papers, where both the questions and the marked answers are available. The student who simply memorises the answer sheet does perfectly on those exact papers and falls apart in the real exam. The one who learns the underlying method does well on questions never seen before. That second kind of learning, doing well on the unseen, is the only kind we care about. It has a name: **generalisation**.

Fitting the training data is easy; any flexible enough model can memorise answers it has already been shown. The whole craft is generalisation, being right about the next case. Keep that distinction between memorising and generalising in mind, because the entire bias-variance section later is built on it.

1.2 Train, validation and test

To measure generalisation honestly we must hold data back. Most people know the train/test split. The piece that is easy to overlook, and easy to misuse, is a third slice in the middle.

Slice	What it is for
Training set	fit the model's parameters
Validation set	choose everything else: which model, which degree, which k , how much regularisation
Test set	estimate real-world performance, spent once, right at the end

The training set fits the parameters. Every other decision, and there are many, has to be made somewhere; if you make those choices by looking at the test set, you have quietly turned it into a second training set. The validation set is where those choices belong. The test set is spent once, like breaking glass in an emergency.

Watch out

Every time you peek at the test set to make a decision, it stops being a test. Later we meet cross-validation, which lets you tune honestly without spending the test set at all. Until then, treat the test set as untouchable.

1.3 Regression or classification

The first real judgement is what kind of answer you want. There is a quick answer and a better one. The quick answer is mechanical: if the thing you are predicting is a number on a scale, that is **regression**; if it is a category, that is **classification**. The better answer is that the target is a decision you make, not a fact of the data. Our same customers table can support “how much will this customer spend?”, which is regression on `monthly_charge`, or “will this customer leave at all?”, which is classification on `churn`. Neither is more correct. They answer different questions.

Choosing the target is already modelling. Get it wrong and the cleverest algorithm in the world will hand you a beautiful answer to a question nobody asked. Decide the decision first.

1.4 The running example, now with two targets

We keep last week’s customers so the data never gets in the way of the method. The builder is unchanged; the only new idea is that we now pick out two columns as things to predict, and reuse the Week 1 `prep` pipeline as the first step of every model.

setup.py

python

```
# df, and the leakage-safe `prep` ColumnTransformer, are exactly as built in Week 1.
num = ["age", "income", "tenure"]          # numeric columns
cat = ["plan", "region", "satisfaction"]    # categorical columns

y_reg = df["monthly_charge"]               # a number on a scale -> regression
y_clf = df["churn"]                        # a yes/no category -> classification
```

About 15% of customers churn. That figure, the **base rate**, looks harmless now. Hold on to it: in the evaluation chapter it is going to make one very popular metric look extremely foolish.

2 Regression

Regression predicts a number. We start here because the machinery is the easiest to see, and because a plain linear model is the honest baseline you should always beat before reaching for anything more elaborate.

2.1 Linear regression

A linear model predicts the target as a weighted sum of the features. Fitting it means choosing the weights that make the predictions miss by as little as possible, measured as total squared error.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

fit by $\min_{\beta} \sum_i (y_i - \hat{y}_i)^2$

the prediction is a weighted sum of the features; we pick the weights β that make the total squared error as small as possible

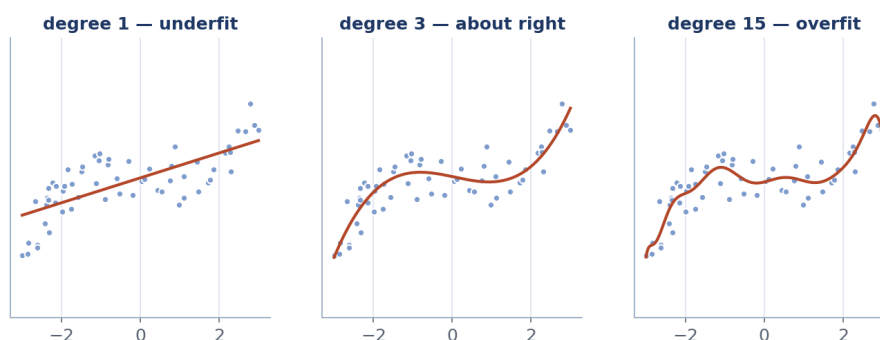
Read this as: multiply each feature by its own weight, add them up along with a baseline β_0 , and that is the prediction. Least squares chooses the weights that minimise the summed squared gaps between prediction and truth. Unlike almost everything later, this has a closed-form solution, so it is instant and completely deterministic.

Read the model, do not just run it

Each coefficient β_j is a claim about the world: holding everything else fixed, one more unit of this feature moves the prediction by this much. So if income comes out with a negative effect on monthly charge, either the world is strange or your data is; both are worth knowing. A model you can interpret is often worth more than a black box you cannot, especially when a decision must be defended.

2.2 Polynomial features and capacity

A straight line cannot bend. When the relationship curves, we can still use the same linear machinery by feeding it powers of the features: x , x^2 , x^3 , and so on. The model is still linear in its weights; it is the inputs that now allow a curve.



The same points fitted at three capacities. Degree 1 cannot bend to the shape and underfits; degree 15 bends to the noise and overfits; degree 3 catches the signal and ignores the rest.

The degree is a capacity dial, and turning it up is not free. The clearest way to see the cost is to score the fit on data it trained on and, separately, on data it did not.

poly.py

python

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

for deg in [1, 3, 15]:
    model = Pipeline([("poly", PolynomialFeatures(deg)),
                      ("lin", LinearRegression())]).fit(x_train, y_train)
    r2_tr = r2_score(y_train, model.predict(x_train))
    r2_te = r2_score(y_test, model.predict(x_test))
    print(f"degree {deg:2d}:  train R2 = {r2_tr:.2f}   test R2 = {r2_te:.2f}")

```

\$ python poly.py

Output

```

degree 1:  train R2 = 0.53   test R2 = 0.51   # too rigid: poor on both
degree 3:  train R2 = 0.79   test R2 = 0.70   # about right
degree 15: train R2 = 0.82   test R2 = 0.65   # train up, test DOWN: overfitting

```

Look at what happens as the degree climbs. At degree 15 the training score keeps rising while the test score falls. That widening gap, brilliant on seen data and worse on unseen, is overfitting made numerical. The model is memorising the noise. Degree 3 is the honest choice here.

2.3 Regularisation: ridge and lasso

If capacity is dangerous, we want a way to keep it in check without abandoning flexible models. That is what regularisation does: it adds a penalty for large coefficients to the thing we minimise, so the model is fitted while being charged for complexity.

$$\begin{aligned} \text{ridge: } & \min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j \beta_j^2 \\ \text{lasso: } & \min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j |\beta_j| \end{aligned}$$

ridge penalises squared coefficients and shrinks them all gently towards zero; lasso penalises absolute values and can push weak coefficients exactly to zero

Read this as: the first term is the old squared error, and the second term is a fine for large weights, scaled by λ (“lambda”), the price of complexity. Ridge uses squared coefficients, so it shrinks everything smoothly and is ideal when features are correlated and noisy. Lasso uses absolute values, and the geometry of that pushes weak coefficients to exactly zero, so it does not just shrink, it **selects**, leaving a simpler model with a shorter story.

When it earns its keep

On a handful of clean features, like ours, regularisation barely changes anything, as the next page shows. Its moment comes with hundreds of correlated features and no idea which matter; there it is the difference between a model and a memoriser.

2.4 Fitting and comparing in code

Because every model sits on top of the same `prep` pipeline, swapping one for another is a one-line change. We split first, fit on the training rows only, and score on the held-out rows.

regress.py

python

```

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

X, y = df[num + cat], df["monthly_charge"]
X, y = X[y.notna()], y.dropna() # keep rows with a known charge
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)

for name, reg in [("linear", LinearRegression()),
                  ("ridge", Ridge(alpha=1.0)),
                  ("lasso", Lasso(alpha=0.1))]:
    model = Pipeline([("prep", prep), ("reg", reg)]).fit(X_tr, y_tr)
    pred = model.predict(X_te)
    print(f"{name:7s} R2 = {r2_score(y_te, pred):.3f} "
          f"RMSE = {np.sqrt(mean_squared_error(y_te, pred)):.2f}")

```

\$ python regress.py

Output

```

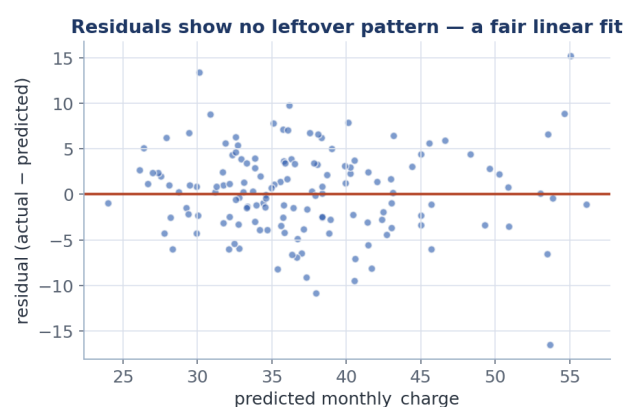
linear  R2 = 0.668  RMSE = 4.74
ridge   R2 = 0.668  RMSE = 4.74
lasso   R2 = 0.670  RMSE = 4.72

```

The three land in almost the same place, and that sameness is the finding, not a disappointment. With few, clean features there is nothing for regularisation to fix, so it correctly does nothing. An R^2 of about 0.67 means the model explains roughly two-thirds of the variation in charge, which for real data is respectable rather than embarrassing.

2.5 Diagnosing a fit: residuals and R^2

A single score never tells the whole story. The residual, the gap between actual and predicted, is where you check whether the straight-line form was appropriate at all. Plot the residuals against the predictions and look at the shape.



Residuals scattered evenly around zero, with no curve and no funnel. That is what a fair linear fit looks like: the model has captured the structure and what remains is noise.

What you want is exactly this: a shapeless cloud around zero. A curve in the residuals would mean the relationship bends and the straight line is missing it; a funnel would mean the error grows with the prediction. Either pattern is a warning that the model form is wrong.

Watch out

A high R^2 sitting on top of a **patterned** residual plot is a red flag, not a triumph. It means real structure is being missed and the number is flattering you. The picture catches what the number hides; this is the same lesson as Anscombe's quartet last week, in a new setting.

3 Classification

Classification predicts a category. We will meet five families of classifier, each thinking about the boundary between classes in a different way. The models are quick; the real difficulty, saved for the next two chapters, is measuring how good they are without fooling ourselves.

3.1 Logistic regression

Despite the name, logistic regression is a classifier. It models the log-odds of the positive class as a straight-line score, then squashes that score into a probability between 0 and 1.

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}}$$

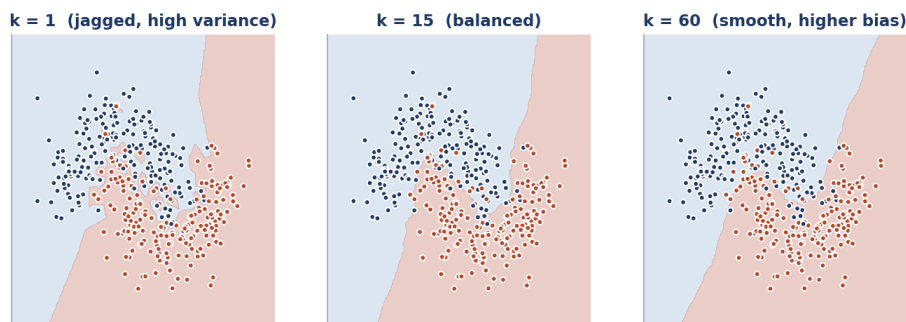
a linear score is passed through the sigmoid, which turns any number into a probability between 0 and 1; a threshold then turns the probability into a yes or no

Read this as: compute a weighted sum of the features, exactly as in linear regression, then feed it through the sigmoid function, whose S-shape maps anything to the range 0 to 1. The result is the model's estimated probability that the case is positive.

The output is a **probability**, not a bare label, and that probability is the raw material for every evaluation tool in the next chapter. The boundary the model fits is one thing; the **threshold** you place on the probability to turn it into a decision is another, and it is entirely yours to set.

3.2 k-nearest neighbours

k-NN does no training at all. It simply remembers every training point, and to classify a new case it looks at the k closest points and takes a vote.



The same data classified at three values of k . At $k = 1$ the boundary is jagged and memorises every point (high variance); at $k = 60$ it is over-smoothed and ignores real structure (high bias); $k = 15$ balances the two.

Here k is the capacity dial: small k memorises and swings wildly, large k smooths everything into mush. Because the whole method rests on distance, the features must be on comparable scales.

Week 1 pays off

If you skip standardisation, income measured in thousands completely swamps age measured in tens, and the “nearest” neighbour is decided by whichever column has the biggest numbers. This is exactly why we standardised last week; k-NN is the model that breaks loudly if you did not.

3.3 Decision trees

A decision tree asks one yes/no question at a time, each split chosen to make the resulting groups purer, measured by Gini impurity or entropy (both just quantify how mixed a group is). You can read the path from root to leaf as a plain sentence, which makes trees the most interpretable model here.

Watch out

Grown without a limit, a tree keeps splitting until it has drawn a box around every single training point: perfect on the training set, useless on new data. Depth is the capacity dial, and a depth cap or pruning is how you hold it back. Interpretability and reliability are not the same virtue; the tree is both the easiest model to read and one of the easiest to overfit.

3.4 Ensembles: bagging and boosting

One of the most useful ideas in the field is that many weak models, combined, beat one strong one. There are two routes.

- **Bagging**, and its famous form the **random forest**, grows many trees on resampled data and random feature subsets. Each tree is wrong in its own direction; averaging them cancels the noise and variance falls.
- **Boosting**, and its modern forms **gradient boosting**, adds small trees in sequence, each one correcting the last one's mistakes. Bias falls, but it is greedy and can overfit, so it needs a gentler hand.

In plain terms

Ensembles win by **diversity**, not brilliance. A hundred mediocre but different trees routinely beat one lovingly tuned expert. The names you will hear in industry are XGBoost and LightGBM; scikit-learn's own `HistGradientBoosting` gives most of the benefit with nothing extra to install, and it is the one we use in the lab.

3.5 Support vector machines

Support vector machines separate the classes by finding the boundary with the widest possible margin, the widest gap you can run between the two groups. Only the points on the edge of that gap, the **support vectors**, actually determine where the boundary sits.

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))$$

find the boundary with the widest margin; the constant C trades a wider margin against tolerating the awkward points that fall inside it

Read this as: the first term makes the margin as wide as possible, and the second term charges a penalty for each point that sits on the wrong side of it. The constant C sets how strict that trade is. The **kernel trick** lets the same idea draw curved boundaries: bend the space so that a straight margin in the new space is a curve in the original, which buys non-linearity without polynomial bookkeeping.

Reach for an SVM when the margin is fairly clean, the sample is modest, or the features vastly outnumber the rows (text classification is the classic case). Skip it on very large datasets, where it is slow, and remember it gives distances from the boundary, not calibrated probabilities.

3.6 Fitting and comparing in code

As with regression, every classifier plugs into the same pipeline, so comparing them is a short loop. Read the table below carefully, because it contains the trap that the whole next chapter is about.

classify.py

python

```

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score)

X, y = df[num + cat], df["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=0, stratify=y)

models = {"logistic": LogisticRegression(max_iter=1000),
          "knn":      KNeighborsClassifier(n_neighbors=15),
          "tree":     DecisionTreeClassifier(max_depth=4, random_state=0),
          "forest":   RandomForestClassifier(n_estimators=300, random_state=0)}

for name, clf in models.items():
    m = Pipeline([("prep", prep), ("clf", clf)]).fit(X_tr, y_tr)
    p = m.predict(X_te)
    pp = m.predict_proba(X_te)[:, 1]
    print(name, accuracy_score(y_te, p), recall_score(y_te, p),
          f1_score(y_te, p), roc_auc_score(y_te, pp))

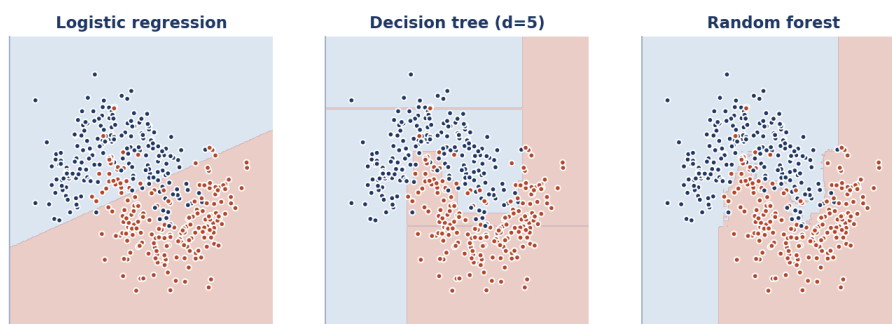
```

```
$ python classify.py
```

Output

	acc	prec	rec	F1	AUC
logistic	0.880	0.600	0.652	0.625	0.949
knn	0.873	0.643	0.391	0.486	0.933
tree	0.900	0.643	0.783	0.706	0.931
forest	0.887	0.636	0.609	0.622	0.956

Every model scores around 87 to 90% accuracy, so on that column alone you would call them interchangeable. Now read the **recall** column. Logistic catches about 65% of the churners, the tree about 78%, but k-NN catches only 39%: it misses six leavers in ten while looking exactly as accurate as the rest. That gap between accuracy and recall is the door into the next chapter.



One dataset, three boundaries. Logistic insists on a straight cut; the tree carves rectangles; the forest blends many trees into a smooth, flexible frontier. The right shape depends on the data's shape.

4 Loss, and the bias-variance trade-off

We now have a shelf of models. This short chapter is the theory that governs all of them at once: what they are really minimising, the two ways they fail, and the single trade-off underneath both.

4.1 Loss functions

The loss function is the definition of “wrong” that the model is actually minimising. Change it and you change what the model cares about, before a single parameter is learned.

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

$$\text{log-loss} = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

squared error, for regression, punishes big misses hardest; log-loss, for classification, punishes confident wrong answers hardest

Read this as: for regression, mean squared error squares each gap, so one huge error hurts far more than several small ones, which lets a few outliers dominate the fit; switching to mean absolute error, $\frac{1}{n} \sum_i |y_i - \hat{y}_i|$, stops that. For classification, log-loss looks at the predicted probability and punishes being confident and wrong far more than being unsure and wrong.

A design choice, not a default

Choosing the loss is really answering “which mistakes can I least afford?”. Answer that first; the optimisation then simply does what you told it to care about, so make sure you told it the right thing.

4.2 Under- and overfitting

There are exactly two ways for a model to fail, and they look the same on the surface.

- **Underfitting** is too rigid: it misses the pattern even on data it has seen. Training and validation scores are both poor, and close together. The cure is more capacity.
- **Overfitting** is too flexible: it learns the noise, so it is excellent on training data and poor on new data. The tell is a wide gap between training and validation scores. The cure is less capacity, or more data.

Both failures show the same symptom, a poor validation score, but the cures are opposite. Add capacity to an overfitting model and you make it worse. So you cannot just react to the score; you must first diagnose which failure you have. The next two sections are how.

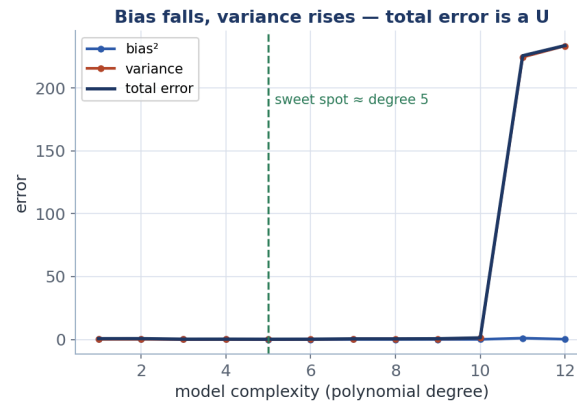
4.3 The bias-variance decomposition

The two failures are the two halves of a single equation. Expected error splits into three parts.

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

bias is error from being too simple; variance is error from being too sensitive to the sample; sigma-squared is irreducible noise no model can beat

Read this as: **bias** is how far wrong the model is on average because it is too simple to represent the truth; **variance** is how much the model jumps around when the training sample changes, because it is too sensitive to it; and σ^2 is the noise floor, the randomness in the world that no model can remove.

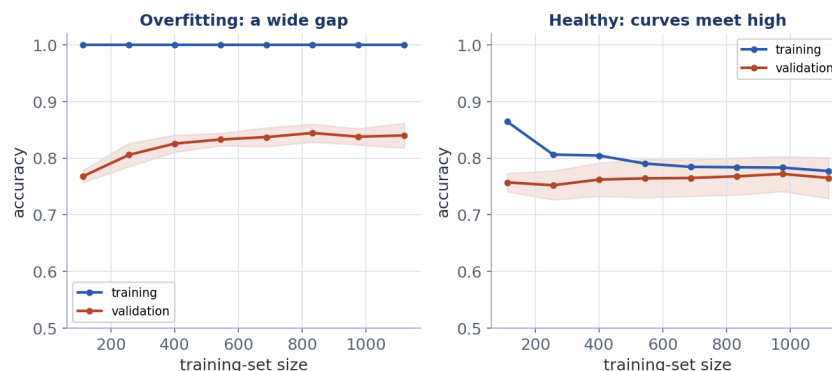


As complexity rises, bias falls but variance rises, and their sum, total error, traces a U. The goal is not zero bias or zero variance; it is the bottom of the U.

Every dial in this booklet, polynomial degree, λ , tree depth, k , the number of trees, is a way of sliding along this curve. They are not five unrelated settings; they are five ways of doing the same one thing, and this is the unifying idea of the whole week.

4.4 Learning curves

The learning curve turns the theory into a practical diagnostic. Plot the training and validation scores against how much training data you have used, and the shape tells you which failure you face.



Two shapes to recognise. A stubborn gap between training and validation (left) is variance, so overfitting; two curves meeting at a low ceiling would be bias, so underfitting.

A gap that will not close means variance: get more data, simplify, or regularise. Two curves that meet but meet low means bias: add capacity or better features, because more data will not help. That second case is where teams waste weeks, pouring in data that was never the problem. The learning curve would have told them in one glance.

5 Evaluation done properly

This is the heart of the week. Everything so far was building models; this chapter is about not lying to yourself with them. If you remember one chapter in six months, make it this one.

5.1 Cross-validation

A single train/test split hands you one number, and one number carries a false air of precision: get lucky with the split and you look good, unlucky and you look poor, and you cannot tell which happened. Cross-validation rotates the held-out slice so that every point is judged exactly once, on a fold it never trained on, giving you a mean and, just as valuable, a spread.

5-fold cross-validation: every row scores on a slice it never trained on

fold 1	val	train	train	train	train
fold 2	train	val	train	train	train
fold 3	train	train	val	train	train
fold 4	train	train	train	val	train
fold 5	train	train	train	train	val

Five-fold cross-validation. Each row trains on four slices and scores on the fifth, so every point is held out exactly once and no point is ever both trained and tested on.

crossval.py

python

```
from sklearn.model_selection import cross_val_score
pipe = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
scores = cross_val_score(pipe, X, y, cv=5, scoring="accuracy") # X, y = churn task
print(scores.round(3))
print(f"mean = {scores.mean():.3f}    std = {scores.std():.3f}")
```

\$ python crossval.py

Output

```
[0.958 0.942 0.892 0.925 0.883]
mean = 0.920    std = 0.029
```

The single split earlier reported 88% accuracy; cross-validation says 92% on average, but with a spread from 88% to 96% across folds. That spread is the honesty: it tells you how much your one number could have wobbled.

Cross-validate the pipeline, not the model

The scaler and imputer must be refit inside each fold, which is exactly what putting them in the pipeline achieves. Fit them once on all the data first and you leak the held-out rows into training through the back door, and the tidy cross-validated score becomes fiction. This is the most common way careful people fool themselves.

5.2 Regression metrics

For a number, three metrics cover most needs, and they have different personalities.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

RMSE is the typical error in the target's own units; R^2 is the share of the variation the model explains, from 0 to 1

Read this as: **RMSE** is a kind of average error, but because it squares before averaging it punishes large misses, and it comes out in the target's units (euros, say), so it is interpretable. **MAE**, the plain average absolute error, is kinder to outliers. **R^2** is unitless and comparable across problems, but that abstraction lets it flatter you on easy data.

When RMSE and MAE disagree sharply, a few big errors are doing the damage: the typical error is small but the tail is nasty. That gap is a clue about where the model fails, not just how much.

5.3 The accuracy trap

Accuracy is the first metric everyone reaches for, and on rare-event problems it is the first to mislead. Suppose fraud runs at about 3% of transactions.

the lazy model

```
prevalence of the positive class      = 0.034
accuracy of "always predict the majority" = 0.966
frauds caught                        = 0
```

A model that simply says “never fraud” scores 96.6% accuracy and catches nothing at all. It is useless, and it beats most first attempts on the accuracy number. Our churn models sit near 88% accuracy, yet we have already seen that number hide a recall spread from 0.39 to 0.78.

Rule of thumb

On any rare-event problem, if someone quotes you accuracy, that should stop you. Ask what a “99% accurate” cancer screen is worth when 1% of patients have cancer: nothing, because “always say healthy” scores 99% and saves no one.

5.4 The confusion matrix

Everything past here is built on four numbers: the two ways of being right and the two ways of being wrong. For our logistic model on the churn test set they are as follows.

Confusion matrix — the two mistakes are not equal

	pred	
	stay	churn
true	stay	117 10
	churn	8 15

The four outcomes for the logistic churn model: 117 true negatives, 15 true positives, and the two errors, 10 false positives and 8 false negatives. The two mistakes are rarely equal in cost.

The diagonal is where the model was right. Off the diagonal are the two errors, and they almost never cost the same. A false negative on a tumour and a false positive on a spam filter are not remotely the same mistake, even though both are simply “one error”.

Before choosing any metric, decide which error hurts more in your problem. Every metric that follows is just a way of caring about one error more than the other; get the cost judgement right and the metric almost chooses itself.

5.5 Precision, recall and F1

Two metrics name the two errors directly, and a third splits the difference.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

precision: of the cases we flagged, how many were real; recall: of the real cases, how many we caught; F1: the harmonic mean of the two

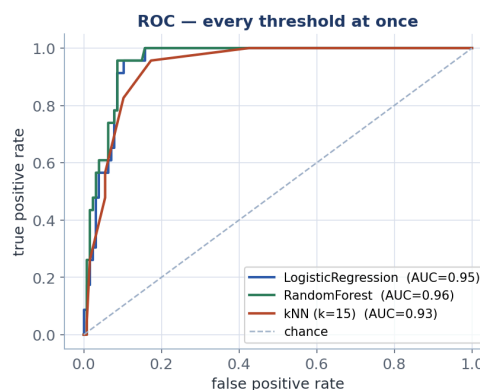
Read this as: **precision** is about not crying wolf, and **recall** is about not missing the real thing. They pull against each other: flag more aggressively and recall rises while precision falls, because more of what you flag is wrong. **F1** is a single compromise number.

Watch out

Do not optimise F1 out of habit. It quietly assumes the two errors matter equally, and we have just established that they usually do not. Optimise the error your problem can least afford, recall for a cancer screen, precision for a spam filter, and let the other float.

5.6 ROC and AUC

Every metric so far depended on a chosen threshold. The ROC curve steps back and sweeps every threshold at once, tracing the trade-off between catching positives and raising false alarms.



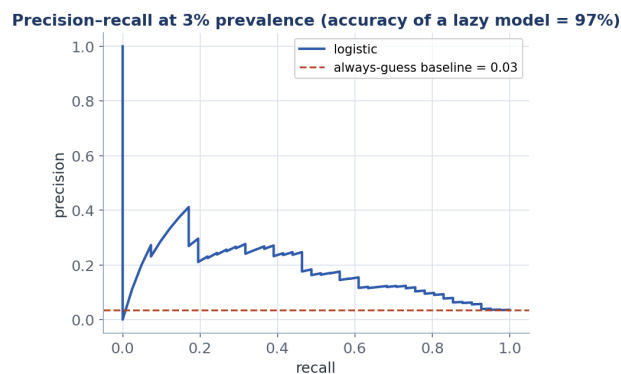
ROC curves for three models on the churn task. The area under each curve, the AUC, measures how well the model ranks positives above negatives; the diagonal is a coin toss.

The AUC has a clean meaning: it is the probability that the model ranks a random positive above a random negative. A half is a coin flip; our forest reaches 0.96, which is strong ranking.

AUC is threshold-free, which makes it ideal for comparing how well models **rank**, and useless for telling you where to actually **cut**. A model can have a lovely AUC and be poor at the threshold you deploy. AUC ranks; it does not decide.

5.7 Precision-recall for rare positives

AUC has a blind spot precisely where the stakes are highest: rare positives. When the positive class is tiny, ROC curves look reassuringly high even for a weak model, because the huge, easy negative class keeps the false-positive rate small no matter what.



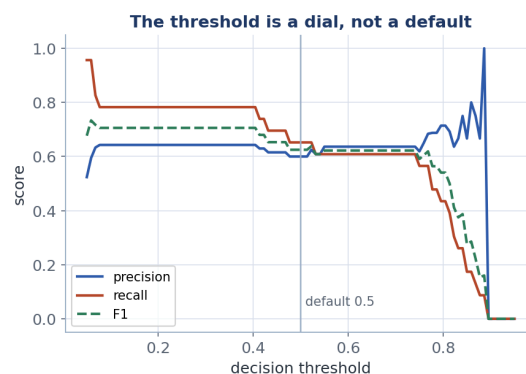
A precision-recall curve at 3% prevalence. Because it ignores the easy negative mass and focuses on the flagged cases, it stays honest where ROC would flatter.

Pick the right curve

For rare positives, read the precision-recall curve and its area (PR-AUC), not ROC-AUC. Same model, same data, genuinely different verdict. Most high-stakes problems, fraud, disease, fault detection, live in exactly this rare-positive regime.

5.8 Choosing the threshold

The default threshold of 0.5 is a convention, not a law. Move it and precision and recall trade off continuously, so the right value depends on which error costs more.



Precision, recall and F1 as the threshold slides. There is no universally right point; the correct one is wherever the costs balance for your problem.

For a cancer screen you push the threshold low, accepting many false positives so as to miss almost no real case. For blocking a customer's card you push it high, because a wrongly blocked customer

is a real cost. Choosing the threshold is a business decision wearing a mathematical costume; own it explicitly rather than letting 0.5 make it for you.

6 Framing real problems

The algorithms are the easy, replaceable part. The judgement that no library supplies is turning a messy real question into a well-posed prediction problem with the right target and the right yardstick.

6.1 The question sets the target, the target sets the metric

There is a chain, and it runs one way: the **question** (what decision are we feeding?) sets the **target** (what should we predict?), which sets the **metric** (how do we measure success given the costs?), which sets the **threshold** (when do we act?). Get that chain right and a plain logistic regression will beat a mis-aimed neural network every time, because the neural network is answering the wrong question beautifully.

6.2 A tour of applications

The same handful of algorithms appears everywhere. What changes, and what you are paid to get right, is the target and the metric, and those come from the domain.

Problem	Frame as	The metric the question demands
Demand forecasting, pricing	regression	RMSE / MAE; over- and under-stocking often cost differently
Credit scoring	classification	AUC and calibration; the cost of a default against lost custom
Churn prediction	classification	recall on likely leavers, or the euro value of retention
Medical diagnosis	classification	recall first; a missed case is the expensive error
Fraud and spam	classification	precision and PR-AUC; a rare-positive problem
Predictive maintenance	classification	recall with enough lead time to act on the warning

6.3 Cost-sensitive thinking: two opposite problems

The clearest way to feel the point is to put two problems side by side.

- **Medical diagnosis.** A missed disease can be fatal; a false alarm means a follow-up test. The costs are wildly lopsided, so recall dominates and you accept more false positives to miss fewer cases.
- **Spam and fraud.** A blocked genuine email or card means a furious customer. So precision matters, and because positives are rare you live on the precision-recall curve.

Same maths, opposite dial

Nothing about the algorithm changed between these two; you could use the identical model. All that flipped was the answer to “which error can we least afford?”, and that single judgement, made in the domain rather than the code, turned the whole evaluation around.

6.4 From a prediction to a decision

A probability is not an action. Someone still has to choose the threshold and decide what happens on the other side of it. A churn probability of 0.7 might mean “telephone them”; 0.4 might mean “send a discount”; below that, do nothing. The model informs; the policy decides. And because data drifts, last year’s threshold quietly rots, which is Week 1’s monitoring loop closing. Evaluate the whole chain that ends in a human decision and a real cost, not just the classifier in the middle of it.

6.5 A framing checklist

Before importing a single estimator, answer these.

1. What **decision** does this feed, and what does a wrong one cost?

2. Is the honest target a **number** or a **category**?
3. Which **error**, false positive or false negative, hurts more here?
4. Is the positive class **rare**? If so, retire accuracy and ROC in favour of precision-recall.
5. Where should the **threshold** sit so the costs balance?

If you can answer these five, you have done the hard 80% of the modelling before touching a model. The thinking is the work; the fitting is the easy part, and it is exactly the judgement the lab and the final project reward.

A Formula cheat-sheet

Every formula from the booklet, gathered in one place.

Quantity	Definition
Linear model	$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$
Least squares	$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2$
Ridge penalty	$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j \beta_j^2$
Lasso penalty	$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j \beta_j $
Logistic (sigmoid)	$P(y = 1 x) = 1/(1 + e^{-(\beta_0 + \beta^\top x)})$
SVM (soft margin)	$\min_{w,b} \frac{1}{2} \ w\ ^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))$
Mean squared error	$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$
Mean absolute error	$\text{MAE} = \frac{1}{n} \sum_i y_i - \hat{y}_i $
Log-loss	$-\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$
Bias-variance	$\mathbb{E}[(y - \hat{f})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$
RMSE	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$
Coefficient of determination	$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$
Precision / Recall	$\text{TP} / (\text{TP} + \text{FP}) \quad \text{TP} / (\text{TP} + \text{FN})$
F1 score	$F_1 = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$

B Running everything yourself

One script, `make_assets.py`, reproduces every number and figure in this booklet. From the project folder, run the three lines below.

terminal**python**

```
python -m venv .venv && source .venv/bin/activate    # make and enter a clean workspace
pip install numpy pandas scikit-learn scipy matplotlib
python make_assets.py                                # writes figs/ and prints every result
```

It prints each labelled result (the regression scores, the classifier comparison, the cross-validation spread, the confusion-matrix counts, the imbalance demonstration, and the polynomial train-versus-test gap) and saves every chart into a `figs` folder. The fastest way to build intuition is to change one thing at a time, for example the regularisation strength, the tree depth, or the value of k , then run it again and watch the numbers and pictures respond.

C Glossary

A quick reference for the jargon used in this booklet, each in one plain sentence.

Term	Plain meaning
Supervised learning	Learning a rule from examples that have both inputs and answers
Generalisation	Being right about new, unseen cases, not just the training ones
Regression / classification	Predicting a number / predicting a category
Feature / target	An input column / the column we predict
Overfitting / underfitting	Learning the noise / being too simple to learn the pattern
Capacity	How flexible a model is; the dial behind bias and variance
Bias / variance	Error from being too simple / error from being too sensitive to the sample
Regularisation	Penalising complexity to control capacity (ridge, lasso)
Ensemble	Combining many models; bagging (forests) and boosting
Cross-validation	Rotating the held-out slice to get an honest score and its spread
Precision / recall	Not crying wolf / not missing the real thing
Confusion matrix	The table of true and false positives and negatives
ROC / AUC	The all-thresholds curve / the area under it, a ranking score
PR-AUC	Area under the precision-recall curve; the honest choice for rare positives
Threshold	The probability cut-off that turns a score into a decision

Going further. G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning* (Python edition, free online). · T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*. · A. Géron, *Hands-On Machine Learning* (3rd ed.). · L. Breiman, “Statistical Modeling: The Two Cultures”, *Statistical Science* 16(3), 2001. · T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot on Imbalanced Datasets”, *PLOS ONE*, 2015. · The scikit-learn User Guide, chapters on Model Evaluation and Cross-validation.